

An Efficient and Smooth Volume Shell Mapping by Nonlinear Ray Traversal of Hierarchical Structures

Mayuka Kuwana
Keio University
Japan
mkuwana@keio.jp

Issei Fujishiro
Keio University
Japan
fuji@ics.keio.ac.jp

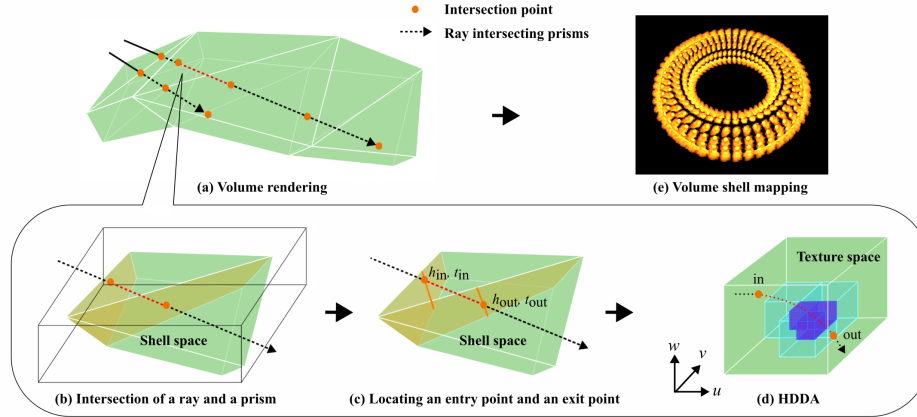


Figure 1: An overview of the proposed method. (a) Volume rendering of all prisms that rays intersect. (b) When a ray intersects with a prism, (c) locate an entry point and an exit point, and then (d) execute Hierarchical Digital Differential Analyzer. (e) Various materials having the effect of transmission and emission can be represented. See the supplementary video as well.

Abstract

Shell mapping is a method of representing microstructures on surfaces by mapping a three-dimensional texture space bijectively into a shell space. This paper introduces a volumetric texture to the texture space for the purpose of representing implicit surfaces and translucent materials. However, conventional approaches often generate artifacts at the boundaries of neighboring tetrahedra due to piecewise-linear approximate mapping. To achieve smooth mapping, we present an approach of mapping rays nonlinearly into the texture space, and we also address efficient sampling by using hierarchical volumetric textures.

CCS Concepts

• Computing methodologies → Volumetric models.

Keywords

shell mapping, rendering, ray tracing, volumetric models, NanoVDB

ACM Reference Format:

Mayuka Kuwana and Issei Fujishiro. 2024. An Efficient and Smooth Volume Shell Mapping by Nonlinear Ray Traversal of Hierarchical Structures. In *SIGGRAPH Asia 2024 Technical Communications (SA Technical Communications '24)*, December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3681758.3697994>

1 Introduction

To enhance the visual quality of three-dimensional (3D) models, various techniques, such as texture mapping, normal mapping, and displacement mapping, have been widely used. Moreover, it is also possible to take advantage of hardware accelerations by applying to simple polygons textures that represent details [Hasselgren et al. 2021]. However, these are inadequate for representing the complex microstructures of surfaces, such as animal hairs or clothing fibers. As such, shell mapping [Porumbescu et al. 2005] defines complex shapes as microstructures by mapping a 3D texture space bijectively into a shell space, which is obtained by extruding an existing triangular polygon (base polygon) along normals at vertices of the base polygon.

In this paper, we introduce volumetric textures to the texture space for the purpose of representing implicit surfaces and translucent materials. One method involves dividing a prism into three tetrahedra and mapping the textures into the shell space using piecewise-linear approximation. Although this method is straightforward, it results in artifacts at the boundaries of neighboring tetrahedra. Meanwhile, nonlinear ray tracing [Ogaki 2023] avoids the tetrahedral decomposition by mapping rays in the shell space into

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SA Technical Communications '24, December 03–06, 2024, Tokyo, Japan
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1140-4/24/12
<https://doi.org/10.1145/3681758.3697994>

the texture space. This paper extends this work to achieve smooth volume shell mapping, without the artifacts associated with the tetrahedral decomposition. In addition, we address efficient volume sampling, referred to as Hierarchical Digital Differential Analyzer (HDDA) [Museth 2014], by utilizing hierarchical volumetric textures in the format of NanoVDB [Museth 2021].

2 Related Work

Shell mapping [Porumbescu et al. 2005] divides a prism into three tetrahedra and maps the texture space into the shell space using the barycentric coordinates of each tetrahedron. The effect of light absorption in participating media can even be simulated by defining the texture space as translucent volumetric textures [Dufort et al. 2005]. With these methods, however, rays in the texture space become C^0 continuous at the boundaries of neighboring tetrahedra, leading to artifacts. This issue becomes particularly serious when the normal at each vertex of a base polygon significantly differs from that of the base polygon or when a base polygon is extruded to a considerable height.

One approach [Jeschke et al. 2007], which solves a cubic equation for the correspondence between the texture space and the shell space at each sampling, eliminates the tetrahedral decomposition, making rays in the texture space C^1 continuous. This could solve the conventional problem of artifacts, but representing the accurate geometry would take much sampling.

Rays mapped into the texture space are nonlinear, and they are formulated as rational quadratic functions [Ogaki 2023]. We extend this method because we can avoid many of the cubic equations required for sampling, as well as the artifacts caused by tetrahedral decomposition.

3 Preliminaries

In this section, we describe a nonlinear ray (Section 3.1) and an acceleration structure for quickly locating intersections of rays and prisms (Section 3.2), both of which stem from [Ogaki 2023].

3.1 The formulation of a nonlinear ray

Given a starting point $\mathbf{o}$, a direction $\boldsymbol{\omega}$, and a distance t , a ray $\mathbf{o} + \boldsymbol{\omega}t$ intersecting the prism, as shown in Fig. 2, can be formulated by:

$$\mathbf{o} + \boldsymbol{\omega}t = (1 - \alpha - \beta)\mathbf{S}_0(h) + \alpha\mathbf{S}_1(h) + \beta\mathbf{S}_2(h), \quad (1)$$

$$\mathbf{S}_i(h) = \mathbf{P}_i + h\mathbf{N}_i, \quad (2)$$

where $\mathbf{S}_i(h)$ ($i = 0, 1, 2$) represents a position between the vertex $\mathbf{P}_i$ of a base polygon and the vertex $\mathbf{P}_i + \mathbf{N}_i$ of the offset polygon, with an interpolation coefficient h ($0 \leq h \leq 1$). $(1 - \alpha - \beta, \alpha, \beta)$ denotes the barycentric coordinates of $\mathbf{o} + \boldsymbol{\omega}t$ in the triangle $\mathbf{S}_0(h)\mathbf{S}_1(h)\mathbf{S}_2(h)$.

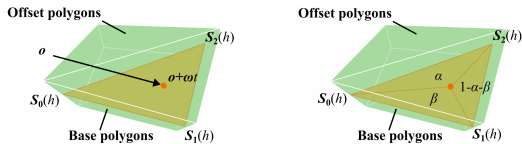


Figure 2: Convert a ray in a world space to the barycentric coordinates of the triangle $\mathbf{S}_0(h)\mathbf{S}_1(h)\mathbf{S}_2(h)$

By solving the two equations obtained from multiplying both sides of Eq. (1) by $\mathbf{e}_0$ and $\mathbf{e}_1$ perpendicular to $\boldsymbol{\omega}$, α and β can be expressed as rational quadratic functions of h :

$$\begin{aligned} \alpha(h) &= \frac{\alpha_2 h^2 + \alpha_1 h + \alpha_0}{d_2 h^2 + d_1 h + d_0}, \\ \beta(h) &= \frac{\beta_2 h^2 + \beta_1 h + \beta_0}{d_2 h^2 + d_1 h + d_0}. \end{aligned} \quad (3)$$

We derive the ray in the texture space from α and β :

$$u = (u_1 - u_0)\alpha + (u_2 - u_0)\beta + u_0, \quad (4)$$

$$v = (v_1 - v_0)\alpha + (v_2 - v_0)\beta + v_0, \quad (5)$$

$$w = (w_{\text{top}} - w_{\text{bottom}})h - w_{\text{bottom}}, \quad (6)$$

where (u_i, v_i) are the uv coordinates at $\mathbf{P}_i$ of the base polygon, and the range of the w coordinate is from w_{bottom} to w_{top} . From Eqs. (3), (4), and (5), the uv coordinates of the ray are:

$$u(h) = \frac{u_2 h^2 + u_1 h + u_0}{d_2 h^2 + d_1 h + d_0}, \quad (7)$$

$$v(h) = \frac{v_2 h^2 + v_1 h + v_0}{d_2 h^2 + d_1 h + d_0}. \quad (8)$$

See [Ogaki 2023] for details on α_i , β_i , and d_i .

3.2 Acceleration structure

Similar to previous attempts, we adopted a two-level Bounding Volume Hierarchy (BVH) [Clark 1976] as an acceleration structure to locate intersections of rays and prisms. Each leaf node of the Bottom Level Acceleration Structures in the BVH contains an Axis-Aligned minimum Bounding Box (AABB) with respect to a prism. Figure 1 (b) shows how to find intersections of a ray and a prism, composed of five faces when intersecting the AABB. Regarding sides that may be non-planar quadrangles, bilinear patches defined by four interpolated points are introduced.

4 Rendering

In this section, we detail the proposed rendering process. As demonstrated in Fig. 1, we focus on the following steps: ray section reduction for volume rendering (Section 4.1), HDDA for efficient nonlinear ray traversals (Section 4.2), and volume rendering to compute final colors (Section 4.3).

4.1 Ray section reduction

We narrow a ray range by finding parameters (u, v, w) at an entry point and an exit point. First, we compute h_{in} , h_{out} , t_{in} , and t_{out} , as explained in Fig. 1 (c). For base triangles, we solve the following cubic equation numerically using a robust version of Newton's method [Yüksel 2022]:

$$\begin{aligned} (\mathbf{I} - \mathbf{S}_0(h), \mathbf{N}(h)) &= 0, \\ \mathbf{N}(h) &= (\mathbf{S}_2(h) - \mathbf{S}_0(h)) \times (\mathbf{S}_1(h) - \mathbf{S}_0(h)). \end{aligned}$$

If an intersection occurs at the boundary or inside a prism, it converges with relatively few iterations [Thonat et al. 2023]. For bilinear patches on sides, we use an efficient method [Reshetov 2019]. We obtain the barycentric coordinates of an entry point and an exit point in $\mathbf{S}_0(h)\mathbf{S}_1(h)\mathbf{S}_2(h)$ using h_{in} , h_{out} , t_{in} , and t_{out} , and finally, (u, v, w) is obtained from Eqs. (4), (5), and (6).

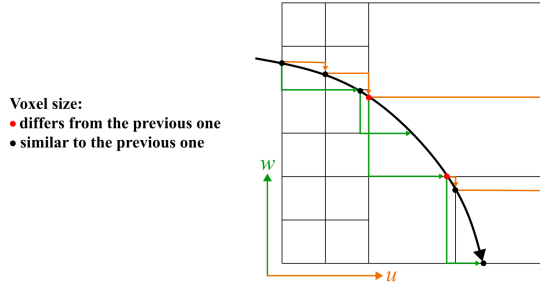


Figure 3: An overview of HDDA. A parameter in the current sampling is updated to leave the voxel. Based on the updated parameter, the other two parameters are computed according to the case division described in Section 4.2.

4.2 HDDA

Sampling nonlinear rays is essential to volume rendering; as such, we traverse by HDDA, as described in Fig. 1 (d). Volumetric textures satisfy the NanoVDB specification, which enables fast data access. HDDA is a generalization of the Bresenham [Bresenham 1965]. The next sampling is determined based on the current sampling and the voxel size, as shown in Fig. 3. When updating u , v , or w , we use Eqs. (7), (8), and (6), respectively. Note that there are cases in which the denominator in Eqs. (3), (7), and (8) is equal to 0 if h is equal at both an entry point and an exit point. To compute robustly, we derive an equation in terms of α and β by eliminating h from Eqs. (1) and (2), resulting in the following conic curve:

$$a\alpha^2 + b\alpha\beta + c\beta^2 + d\alpha + e\beta + f = 0. \quad (9)$$

See Section 2 of the supplemental material for details of these coefficients. We divide the issue into three cases using Eqs. (4), (5), (6), (7), (8), and (9) to find the other two parameters:

(1) if u is updated:

v is found by substituting β , obtained from solving Eqs. (4) and (9), into Eq. (5), and w is derived from substituting h , found from the following equation, into Eq. (6):

$$(ud_2 - u_2)h^2 + (ud_1 - u_1)h + (ud_0 - u_0) = 0$$

(2) if v is updated:

u is found by substituting α , obtained from solving Eqs. (5) and (9), into Eq. (4), and w is derived from substituting h , found from the following equation, into Eq. (6):

$$(vd_2 - v_2)h^2 + (vd_1 - v_1)h + (vd_0 - v_0) = 0$$

(3) if w is updated:

u, v are derived from substituting h , found from Eq. (6), into Eqs. (7) and (8).

There is only one solution to these quadratic equations provided a ray direction. The pseudo-code of HDDA is given in Algorithm 1. At the first sampling, u , v , and w are updated to be outside the current voxel, and all candidates for the next sampling are computed. The selected next sampling is the one with the minimum distance from the current sampling. If the current voxel size differs from the previous one (red dots in Fig. 3), we find all candidates. Conversely,

Algorithm 1: Pseudo-code of HDDA algorithm, which efficiently samples a space filled with heterogeneous media.

Input: candidates: $pos[updatedParam].component$

Output: transmittance: $transmittance$, radiance: L
 $axis \leftarrow all$

Initialize($u_{next}, v_{next}, w_{next}, val$)

while $t < t_{out}$ **do**

if $axis = u$ **or** all **then**

$pos[u].u \leftarrow u_{next}$

 update $pos[u].v$ and $pos[u].w$ according to (1)

else if $axis = v$ **or** all **then**

$pos[v].v \leftarrow v_{next}$

 update $pos[v].u$ and $pos[v].w$ according to (2)

else if $axis = w$ **or** all **then**

$pos[w].w \leftarrow w_{next}$

 update $pos[w].u$ and $pos[w].v$ according to (3)

 SelectTheClosestCandidate(pos)

$axis \leftarrow GetSelectedAxis()$

$transmittance, L \leftarrow VolumeRendering(val)$

$val \leftarrow GetValue(axis)$

if voxel size is different from the previous one **then**

$axis \leftarrow all$

 UpdateParam($axis$)

if the size is the same, we find the next candidate only for the selected candidate.

4.3 Volume rendering

Figure 1 (a) shows volume rendering of every shell that a single ray intersects. First, the ray range is narrowed by closest-hit shaders, and then, in any-hit shaders, transmittance and radiance are computed using HDDA. In this paper, we consider only absorption, out-scattering, and emission, except for in-scattering.

5 Results

In this section, we empirically evaluate our method by assessing its quality and performance. All experiments were conducted on a computer with an NVIDIA GeForce RTX 4080, an Intel Core i7-13700F 2.10 GHz CPU with 64 GB of RAM, and a Windows 11 x64 operating system. We implemented the CPU renderer using Embree [Wald et al. 2014] and the GPU one using Optix [Parker et al. 2010].

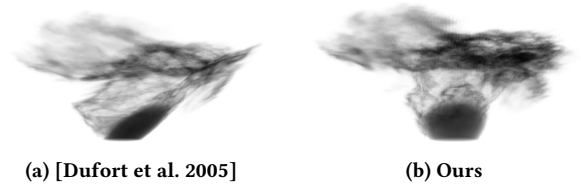


Figure 4: Quality comparison between the tetrahedral linear approximation method (a) and the nonlinear method (b).

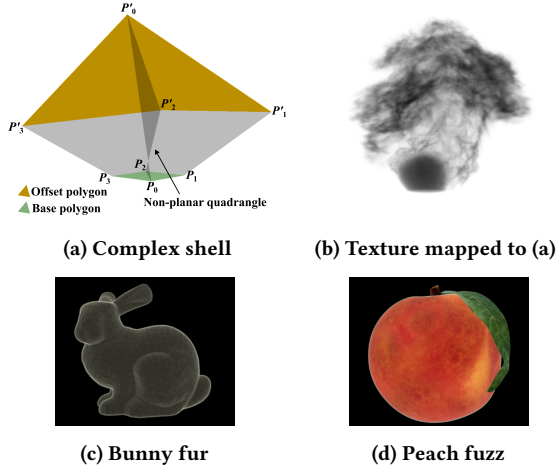


Figure 5: Examples of volume shell mapping.

Table 1: Comparison of the rendering time in scenes.

	GPU/CPU	[Dufort et al. 2005]	Ours	Non-hierarchy
Box	GPU	0.173s	0.298s	0.483s
	CPU	0.546s	0.831s	1.085s
Torus	GPU	0.498s	1.136s	1.503s
	CPU	0.829s	1.156s	1.305s
Peach	GPU	0.785s	1.684s	2.358s
	CPU	0.738s	0.936s	1.109s
Stanford	GPU	0.798s	1.575s	2.078s
Bunny	CPU	0.724s	0.994s	1.107s

5.1 Quality evaluation

First, we compared our method with the tetrahedral decomposition method using the volumetric texture in [JangaFX Software 2024]. Shell spaces composed of two prisms are shown in Fig. 4 (a) and (b). It was observed that the distortion shown in shell spaces (a) caused by tetrahedral decomposition disappeared in our method (b). Figure 5 (a) and (b) show an example of mapping the volumetric texture to complex shell spaces. It can be confirmed that mapping is smooth, even when sides are non-planar quadrangles. In addition, Fig. 1 (e) is a case of instancing along torus surfaces, accounting for both heterogeneous transmission and emission in participating media. As examples of application, a furry Stanford Bunny and a fuzzy peach [printable_models 2024] are demonstrated in Fig. 5 (c) and (d), respectively. See Section 5 of the supplemental material for magnified views of these examples.

5.2 Performance evaluation

Table 1 shows the rendering time when the volumetric texture in [JangaFX Software 2024] is mapped and rendered at a resolution of $1,024 \times 512$ pixels. The number of voxels in the texture is 136,894. In all scenes, the rendering time of our method is longer than that of the tetrahedral decomposition method. This indicates that the

relationship between the rendering time and the quality described in Section 5.1 is a trade-off, but the overhead remains relatively small. It also demonstrates the rendering time is shorter when using hierarchical structures, which suggests the usefulness of HDDA. Note that the scenes are not compared to each other here because the cache in NanoVDB may reduce read access depending on the visibility of prisms. In addition, we still have room to improve HDDA, as many flow control instructions in HDDA (if, switch, for, while, etc.) cause warp divergence in the GPU implementation.

6 Conclusion

This work achieved smooth volume shell mapping by tracing non-linear rays in the texture space. Although the performance was slightly degraded, we proposed to make ray traversals as efficient as possible using hierarchical volumetric textures. We also demonstrated the usefulness of our method with several examples and evaluation experiments. However, as described in Section 4.3, we ignored in-scattering, which is a difficult task because incoming rays are nonlinear and phase functions are distorted.

Acknowledgments

This work has been partially supported by Grant-in-Aid for Scientific Research (A) JP21H04916.

References

- Jack E. Bresenham. 1965. Algorithm for Computer Control of a Digital Plotter. *IBM Systems Journal* 4, 1 (1965), 25–30.
- James H. Clark. 1976. Hierarchical Geometric Models for Visible Surface Algorithms. *Commun. ACM* 19, 10 (1976), 547–554.
- Jean-Francois Dufort, Luc Leblanc, and Pierre Poulin. 2005. Interactive Rendering of Meso-Structure Surface Details Using Semi-Transparent 3D Textures. In *Proceedings of the Vision, Modeling, and Visualization*. 399–406.
- Jon Hasselgren, Jacob Munkberg, Jaakko Lehtinen, Miika Aittala, and Samuli Laine. 2021. Appearance-Driven Automatic 3D Model Simplification. In *Eurographics Symposium on Rendering*. 85–97.
- JangaFX Software. Last access date: October 09, 2024. *Free VDB Volumes Created with EmberGen: Simple Fire*. <https://jangafox.com/software/embergen/download/free-vdb-animations/>
- Stefan Jeschke, Stephan Mantler, and Michael Wimmer. 2007. Interactive Smooth and Curved Shell Mapping. In *Proceedings of the 18th Eurographics Conference on Rendering Techniques*. 351–360.
- Ken Museth. 2014. Hierarchical Digital Differential Analyzer for Efficient Ray-Marching in OpenVDB. In *ACM SIGGRAPH 2014 Talks*. 40:1–40:1.
- Ken Museth. 2021. NanoVDB: A GPU-Friendly and Portable VDB Data Structure for Real-Time Rendering and Simulation. In *ACM SIGGRAPH 2021 Talks*. 1:1–1:2.
- Shinji Ogaki. 2023. Nonlinear Ray Tracing for Displacement and Shell Mapping. In *SIGGRAPH Asia 2023 Conference Papers*. 93:1–93:10.
- Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. 2010. OptiX: A General Purpose Ray Tracing Engine. *ACM Transactions on Graphics* 29, 4 (2010), 66:1–66:13.
- Serban D. Porumbescu, Brian Budge, Louis Feng, and Kenneth I. Joy. 2005. Shell Maps. *ACM Transactions on Graphics* 24, 3 (2005), 626–633.
- printable_models. Last access date: October 09, 2024. *Peach 3D Model*. <https://free3d.com/3d-model/-peach--306929.html>
- Alexander Reshetov. 2019. *Cool Patches: A Geometric Approach to Ray/Bilinear Patch Intersections*. 95–109.
- Theo Thonat, Iliyan Georgiev, François Beaune, and Tamy Boubekeur. 2023. RMIP: Displacement Ray Tracing via Inversion and Oblong Bounding. In *SIGGRAPH Asia 2023 Conference Papers*. 12:1–12:11.
- Ingo Wald, Sven Woop, Carsten Benthin, Gregory S. Johnson, and Manfred Ernst. 2014. Embree: A Kernel Framework for Efficient CPU Ray Tracing. *ACM Transactions on Graphics* 33, 4 (2014), 143:1–143:8.
- Cem Yuksel. 2022. High-Performance Polynomial Root Finding for Graphics. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 5, 3 (2022), 27:1–27:15.